

PRD — Mobile-first AI Fitness Self-Assessment (MVP)



Pitch: A mobile-first AI platform that allows athletes in remote India to self-assess standard fitness tests (like vertical jumps and sit-ups) using on-device computer vision, ensuring authentic data reaches SAI without requiring expensive infrastructure.

1) Executive Summary

- **Product / Project name:** Khel Mitra
- **Mission (philosophy): Democratize talent discovery through transparency** — ensure that a promising athlete in a remote village can be *seen, verified, and trusted* by the same national standard used in city academies.
- **What is broken today (trust gap):** Rural and low-infrastructure athletes are often **invisible** to formal pathways because officials cannot reliably verify talent **without travel, supervision, or expensive facilities**. This creates an access and trust gap where genuine performance is difficult to authenticate and compare.
- **Who this serves:**
 - Primary: Athletes and coaches in remote / low-infrastructure settings
 - Secondary: SAI evaluators and program managers who need reliable fitness data
- **Target users:**
 - Primary: Athletes and coaches in remote / low-infrastructure settings
 - Secondary: SAI evaluators and program managers who need reliable, auditable fitness evidence
- **Proposed solution (MVP):** A mobile-first app that guides athletes through standardized fitness tests and acts as a **digital notary** using on-device

computer vision.

- It does not only *score* performance. It creates a **Package of Truth: AI Score + Video Evidence + Metadata + Integrity Checks**, securely submitted to SAI for review.
- This provides **transparency for both sides**: the athlete can prove effort and performance, and the SAI official can verify the evidence without ambiguity.
- **Standardization (non-negotiable)**: The AI enforces a consistent protocol so that a **push-up in a remote village is judged by the exact same strict standard** as one in a city academy.
- **Goals (measurable)**:
 - Enable athletes to complete at least 2–3 standard tests end-to-end from a basic smartphone.
 - Produce verifiable results as a **Package of Truth** that supports remote decision-making and spot-check audits.
- **Success metrics**:
 - Completion rate per test
 - Agreement rate vs supervised validation (spot checks)
 - Reduction in unverifiable / disputed submissions
 - Fraud / anomaly detection rate
 - Submission-to-review time
 - App crash-free sessions and on-device inference latency
- **Constraints**:
 - Intermittent connectivity in rural areas
 - Low-to-mid range Android devices
 - Data privacy for minors and athletes
 - Need for tamper resistance without expensive hardware

2) Problem Statement

SAI's ability to discover and validate athletic talent at scale is constrained by an **evidence problem**.

- **The Trust Gap:** Outside formal academies, authentic performance is hard to verify. Without supervised testing, officials must rely on inconsistent reporting, informal claims, or travel-intensive evaluations. As a result, many rural athletes remain **unseen**.
- **The Cost of Opacity:** When results are not auditable, selection processes become slower, more expensive, and more vulnerable to dispute. This reduces confidence for athletes and officials and limits fair access to opportunity.
- **The Need:** SAI requires a **standardized, transparent, remotely verifiable** method to capture fitness evidence that is comparable across districts and conditions, and usable under low connectivity and low infrastructure.

☐ Scope Lockdown (MVP Tests Only):

- Vertical Jump:
 - Metric: Jump Height (cm) based on Hip/Ankle displacement.
 - Anti-Cheat: Must start from static position (detected by AI).
- Sit-Ups (1 Minute):
 - Metric: Repetition Count.
 - Strict Form: AI counts a rep ONLY if:
 - State A: Shoulders touch ground.
 - State B: Elbows pass knees.
- Excluded: Running (GPS fraud risks), Push-ups (Camera angle issues on floor), Plank (Hard to measure form).

3) The "No" List (Out of Scope)

- ☐ **No iOS App:** Focus strictly on **Android** (India's rural market is 95% Android). Use Flutter or Native Android.

- ☐ **No Real-time “Endurance Run” Tracking:** GPS fraud is too easy and battery-intensive for V1. Focus on stationary tests first (Vertical Jump, Sit-ups).
- ☐ **No Complex “Gamification”:** Leaderboards are fine; complex 3D badges and avatars are a waste of time.
- ☐ **No “Live” Streaming:** Video is heavy. We analyze on-device or upload compressed chunks. We do NOT stream live.

3) Core User Flow

Primary user persona: Remote athlete with an Android phone, limited data plan, and minimal coaching supervision.

Trigger: Athlete needs to submit standardized fitness scores for selection, progress tracking, or periodic reporting.

Master Specification (Engineering Blueprint)

A) Tech Stack (Exact Packages)

- ☐ **Mobile (Android-first):**
 - ☐ **Flutter**
 - ☐ Camera: `cameraawesome`
 - ☐ Must support **simultaneous** camera preview + frame stream for analysis
 - ☐ Must support **recording** each attempt to a local file
 - ☐ Pose/vision: `google_mlkit_pose_detection` (**Stream mode**) for live landmark extraction
 - ☐ Offline queue + local persistence: `sqflite`
 - ☐ Store attempts, scores, and sync state locally so submissions survive app restarts
 - ☐ Networking: `dio`
 - ☐ Implement retries with exponential backoff
 - ☐ Resume uploads when connectivity returns

- ☐ Timeouts and clear error states
- ☐ **Minimum Device Requirements (Hard Limits):**
 - **Android Version: Android 10 (API Level 29)+**
 - *Reason:* Mandatory for performant NPU access (NNAPI) needed by ML Kit. Android 7/8 will be too slow for real-time jump tracking.
 - **RAM: Minimum 3GB RAM.**
 - *Reason:* Running OS + Camera + ML Model + Video Recorder simultaneously requires ~1.2GB of heap. 2GB devices will crash.
 - **Architecture: arm64-v8a** (64-bit) only. 32-bit is not supported for high-performance ML.
- ☐ **Media Quality Constraints (Hard Limits):**
 - **Resolution: 480p (SD) or 720p (HD) max.**
 - *Reason:* 480p is sufficient for pose analysis and keeps file size small (~2MB/min).
 - **Frame Rate: 30 FPS fixed.**
 - *Reason:* 60 FPS doubles the file size and NPU load without adding accuracy.
 - **Bitrate: Cap at 1.5 Mbps.**
 - **Audio: MUTED/DISABLED** for recording.
 - *Reason:* We don't need to hear them grunting. Removes 20% of file size.
- ☐ **Backend:**
 - ☐ **Express.js**
 - ☐ Validation: `zod` for request/response schema validation
 - ☐ Upload handling: `multer` for multipart video uploads
- ☐ **Evidence Storage:**
 - ☐ **Supabase Storage** for video evidence (best-attempt only)

❑ MVP Admin Portal (Flutter Web):

- **Tech:** Flutter Web (hosted on Firebase Hosting or Vercel).
- **Why:** Reuses the existing Dart codebase (Models, API Logic, UI Widgets) to save development time.
- **Views Required:**
 1. **Dashboard:** A standard `DataTable` widget listing submissions (Name, Score, Status).
 2. **Verification Mode:** A detail screen reusing the `VideoPlayer` widget to review the evidence.
- **Actions:** Buttons to API endpoints: `POST /approve` , `POST /reject` , `POST /request_retake` .
- **Security:** This web build must be protected by the `role: 'OFFICIAL'` check on login.

B) Development Environment & Workflow (CRITICAL)

- **Constraint:** Do **NOT** use Android Emulators for ML/Camera testing.
 - *Reason:* Emulators lack NPU access, causing the pose detection to run at <2 FPS (unusable).
- **Mandatory Setup (Physical Device via USB):**
 - All development must be done on a physical Android device connected via USB-C.
 - **Networking:** Use `adb reverse tcp:3000 tcp:3000` to tunnel backend traffic.
 - **API Config:** Set the API Base URL to `http://localhost:3000` (The phone treats the tunnel as local).
- **Hardware Requirement:** Minimum Android 10 device for performant NPU access during debugging.

C) Development Environment & Workflow (CRITICAL)

- **Constraint:** Do **NOT** use Android Emulators for ML/Camera testing.

- *Reason:* Emulators lack NPU access, causing the pose detection to run at <2 FPS (unusable).
 - **Mandatory Setup (Physical Device via USB):**
 - All development must be done on a physical Android device connected via USB-C.
 - **Networking:** Use `adb reverse tcp:3000 tcp:3000` to tunnel backend traffic from phone to laptop.
 - **API Config:** Set the API Base URL to `http://localhost:3000` in the Flutter app (The phone treats the tunnel as local).
 - **Hardware Requirement:** Minimum Android 10 device for performant NPU access during debugging.
 - **Docker Strategy:**
 - **Backend:** Run Node.js + Postgres inside Docker Containers.
 - **Mobile:** Run Flutter directly on the Host Machine (Laptop) to access USB hardware.
-

3. Authentication & Onboarding Strategy (Unified)

3.1 Athlete Login (Mobile — OTP First)

- **Technology:** Firebase Authentication (Phone Provider).
- **OTP Flow:**
 - **Step 1:** User enters phone number.
 - **Step 2:** Auto-verify OTP (using Android SMS Retriever API).
 - **Step 3 (Routing Logic):**
 - After Firebase auth succeeds, the client sends the ID Token to the backend.
 - **Backend Check:** Query `users` table by `firebase_uid`.
 - **If New:** Create row (`role: ATHLETE` , `is_profile_complete: false`) → Redirect to **Profile Setup**.

- **If Existing (`is_profile_complete: false`):** Redirect to **Profile Setup**.
- **If Existing (`is_profile_complete: true`):** Redirect to **Home Dashboard**.
- **Constraint:** 1 Account per Phone Number. Backend enforces uniqueness.

3.2 SAI Official Login (Web Dashboard)

- **Technology:** Firebase Authentication (Email/Password Provider).
- **No Public Sign-up:**
 - **Constraint:** No "Create Account" button on the web portal.
 - **Creation:** Admin accounts are pre-created by the Super Admin via the Firebase Console or a protected script.
- **Login Flow:**
 - **Step 1:** Official logs in on the Web Portal using Email/Password.
 - **Step 2:** Firebase returns an ID Token.
 - **Step 3:** Portal sends this Token to the Backend for every request.

3.3 The "One Token" Backend Security

- **Core Principle:** The backend does not manage passwords. It only verifies Firebase Tokens.
- **Middleware Logic (Node.js + firebase-admin):**
 - **Step 1 (Verify):** Middleware intercepts the `Authorization: Bearer <token>` header and verifies it with Firebase.
 - **Step 2 (Identify):** Middleware extracts the `uid` and finds the corresponding user in the Postgres `users` table.
 - **Step 3 (Authorize):**
 - If `user.role == 'OFFICIAL'`, allow access to Admin Routes.
 - If `user.role == 'ATHLETE'`, deny access to Admin Routes.

3.4 Database Schema Additions

- **Users Table Updates:**

- `firebase_uid` (String, Primary Key/Unique Index) — *Replaces proprietary IDs.*
 - `phone_number` (String, Unique, Nullable for officials).
 - `email` (String, Unique, Nullable for athletes).
 - `role` (Enum: `ATHLETE`, `OFFICIAL`, `SUPER_ADMIN`).
 - `is_profile_complete` (Boolean, default `false`).
 - **Submission Status Enum (The Workflow):**
 - **Field:** `status` (Enum: `PENDING`, `APPROVED`, `REJECTED`, `RETAKE_REQ`).
 - **The "Retake" Logic:**
 - **Action:** When Official clicks "Request Retake," set `status = RETAKE_REQ` and add `admin_comment` (e.g., "Camera was too shaky").
 - **App Behavior:** On Home Load, check `GET /latest_submission`. If `status == RETAKE_REQ`, **Unlock** the "Start Test" button and show the `admin_comment` in a red banner.
-

B) Data Collection Rules (User Profile = Required)

- ☐ The app must require a completed **User Profile** before any test can begin.
 - ☐ **User Profile fields (minimum):**
 - ☐ **Full Name** (string)
 - ☐ **Date of Birth (DOB)** (date) — **critical** for age category and applicable standards
 - ☐ **Gender** (select) — **critical** for standards
 - ☐ **Height in CM** (number) — **critical** for AI calibration
 - ☐ The backend must reject any test submission missing any of the above fields.
-

C) "3-Attempt Smart Sync" Logic (Step-by-step Algorithm)

Stage 0 — Preconditions

- ☐ User is authenticated.

- ☐ User Profile is complete (Full Name, DOB, Gender, Height CM).
- ☐ Device permissions granted: Camera + Microphone + Storage (as needed).
- ☐ **Storage Safety Check (Crucial):**
 - **Step:** Before the "Start Test" button enables, the app must check `getAvailableDiskSpace()`.
 - **Rule:** If free space < **500MB**, disable the button and show alert: *"Phone storage full. Please delete 500MB of photos/videos to continue."*
 - **Rationale:** 3 videos x 50MB buffer + temp files requires safe headroom.

Stage 1 — Calibration (Pixel-to-CM Lock)

- ☐ App starts camera with `camerawesome` and begins ML Kit pose detection in **Stream mode**.
- ☐ App instructs user to stand at the required distance and hold still.
- ☐ **Anti-Cheat: Sensor Fusion (The "Static Phone" Guard):**
 - **Tech:** `sensors_plus` package (Flutter).
 - **Logic:** During the entire attempt (from "Start" to "Stop"), monitor the device **Accelerometer** and **Gyroscope**.
 - **Threshold:** If total acceleration variance > **0.5 m/s²** (indicating the phone was held or tilted), **INVALIDATE** the attempt immediately.
 - **Error Message:** "Test Failed: Phone must be placed against a static object (wall/rock). Do not hold the phone."
- ☐ **Calibration: The "Ground Truth" Zero State:**
 - **For Sit-ups:** User must lie flat for 3 seconds before the timer starts.
 - *AI Action:* Record `Shoulder_Y` and `Hip_Y` to define the dynamic `Ground_Plane_Y`.
 - *Counting Rule:* Rep is valid ONLY if `Shoulder_Y` returns to `Ground_Plane_Y` ($\pm 5\%$ tolerance).
 - **For Vertical Jump:**
 - *Constraint:* User enters Height.

- *Mitigation:* We cannot fix the "Lying about Height" in V1 without reference objects. **Risk Accepted for MVP.** However, we strictly lock the `Pixel-to-CM` ratio only when the AI detects `PoseLandmarkType.leftHeel` is static on the ground for 3 seconds.

☐ **Motion gate:** if user moves beyond a small threshold during calibration (landmark jitter or position delta above threshold), calibration must **pause** and require re-lock.

☐ Only after calibration is locked, the attempt loop can start.

Stage 2 — The Attempt Loop (Exactly 3 Attempts)

☐ The user gets **exactly 3 attempts**.

☐ For each attempt `i ∈ {1,2,3}` :

☐ App displays: Attempt `i/3` .

☐ App starts recording video locally (attempt-scoped file).

☐ App runs pose detection continuously in Stream mode during the attempt.

☐ App computes the attempt score using the locked pixel-to-cm ratio and the test's scoring logic.

☐ App ends recording when the attempt ends (auto or user-confirmed depending on test protocol).

☐ App writes an immutable local record to `sqlite` :

☐ `attempt_id` (uuid)

☐ `submission_session_id` (uuid for the 3-attempt set)

☐ `attempt_index` (1, 2, or 3)

☐ `test_id`

☐ `timestamp_start`, `timestamp_end`

☐ `pixel_to_cm_ratio` (locked value)

☐ `score_value`, `score_unit`

☐ `local_video_path`

- ☐ `sync_state = PENDING`
- ☐ **User restriction:** user cannot delete, discard, or overwrite a "bad" attempt.
- ☐ **ML Failure Handling (The "Blackout" Rule):**
 - **Scenario:** `google_mlkit` returns empty pose list (User lost).
 - **Tolerance:** Allow up to **5 consecutive dropped frames** (approx 0.16s).
 - *Action:* Interpolate the position based on previous velocity.
 - **Failure:** If > 5 frames are dropped:
 - **Vertical Jump: INVALIDATE** the specific attempt. (Reason: We missed the peak).
 - **Sit-ups: PAUSE** the counter. Show UI Overlay: "I cannot see you."
Resume only when skeleton is re-acquired.

Stage 3 — Smart Selection (Best Score)

- ☐ After attempt 3 completes, the app selects the **Best Score** of the 3 attempts.
- ☐ Define `best_attempt` deterministically (e.g., max score_value for distance/height tests).
- ☐ Persist `best_attempt_id` in the session record.
- ☐ The "Glitch Guard" (Variance Check):
 - **Logic:** Before selecting the "Best" score, compare it to the "Median" of the other attempts.
 - **Rule:** If `Best_Score > (Median_Score * 1.5)`, flag the session as **SUSPICIOUS**.
 - *Example:* Jump 1: 30cm, Jump 2: 32cm, Jump 3: **90cm** (Glitch).
 - *Action:* In this case, do **NOT** auto-delete the other videos. Upload **ALL 3 videos** for manual review so the official can see the error.
- **Normal Case:** If variance is low (<50%), proceed with "Upload Best Only" to save bandwidth.

Stage 4 — Smart Sync Upload (Bandwidth-optimized)

- ☐ Upload is resilient to offline conditions.

- ☐ If offline, enqueue and retry using `dio` until success.
- ☐ **Upload payload rules:**
 - ☐ Upload **ALL 3 attempt scores** as JSON in a single session payload for consistency tracking.
 - ☐ Upload **ONLY the video of the BEST attempt** to Supabase Storage.
- ☐ **Backend flow (minimum):**
 - ☐ `POST /submissions` accepts the session JSON (3 attempts + best_attempt_id) and validates with `zod`.
 - ☐ `POST /submissions/:id/evidence` accepts the best-attempt video via `multipart`.
 - ☐ Backend returns immutable IDs and server timestamps.
- ☐ **Post-success cleanup:**
 - ☐ After server confirms BOTH JSON session + best video upload success:
 - ☐ Mark all 3 local attempts as `SYNCED` in `sqlite`.
 - ☐ Auto-delete the **other 2 local videos** (non-best) to save storage.
 - ☐ Keep the best video locally only if required for audit; otherwise delete after confirmed remote storage.
- ☐ **The "Orphan" Sweeper:**
 - **Problem:** If JSON uploads but Video upload is interrupted, the submission is incomplete.
 - **Client-Side Fix:** On App Restart, check for local submissions marked `SYNCED` where the video file still exists. Retry the video upload immediately.
 - **Server-Side Fix:** A nightly Cron Job that flags submissions older than 24 hours with `status: PENDING` and `video_url: NULL` as **"Upload Failed"** so admins don't waste time reviewing them.
- ☐ **Smart Cleanup (Conditional Deletion):**
 - **Logic:**
 - IF `is_suspicious == FALSE` : Auto-delete non-best videos after successful JSON sync.

- IF `is_suspicious == TRUE` : **RETAIN ALL 3 VIDEOS.**
 - Do NOT delete anything until the server explicitly returns a "Case Closed" status (future feature) or user manually clears storage.
 - *Reason:* The admin needs to compare Attempt 1 (normal) vs Attempt 3 (glitch) to prove it was a system error vs. fraud.

6) Timeline

Phase	Dates	Milestone / Deliverable	Owner	Notes / Dependencies
Discovery	Week 1–2	Test protocols locked + MVP scope agreed	PM	Confirm initial test list (e.g., sit-ups, vertical jump)
Prototype	Week 3–4	On-device CV prototype with offline flow	ML + Mobile	Target device benchmarks and failure modes
Build	Week 5–8	MVP app + backend + reviewer dashboard	Eng	Evidence packaging, upload queue, auth
Validation	Week 9–10	Pilot with spot-checked supervised tests	Ops	Measure agreement rate, iterate thresholds
Launch	Week 11–12	Controlled rollout + monitoring	Eng + Ops	Rollback plan, support playbook



MVP acceptance checklist (quick):

- ☐ Athlete can complete at least 2 tests end-to-end on a low/mid Android device
- ☐ Results include evidence artifacts and integrity checks
- ☐ Submissions work with intermittent connectivity (queue + retry)
- ☐ Reviewer can accept/reject/retake with an audit trail